Your First ZipperGen Workflow

Draft a reply, ask a person, then send

The ZipperGen Authors

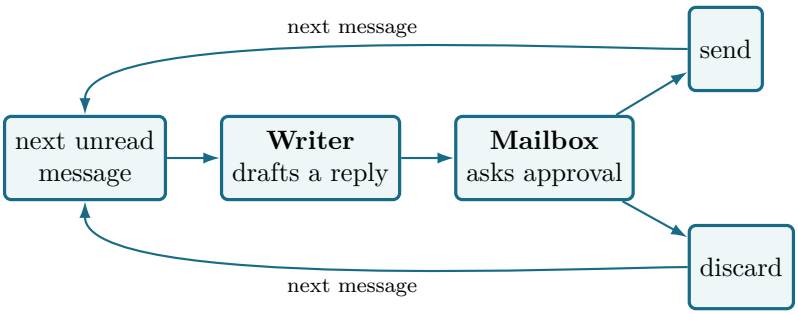
Tutorial edition: 31 August 2026

Abstract

You will build a small application that watches a mailbox, asks a language model to draft a reply to each message, and asks *you* to approve it before anything is sent. It runs continuously as a service, and by the end of the tutorial it is running on a server. It is short enough to read in one go, but it already contains everything that makes ZipperGen worth using: two participants, a model call, an explicit human decision, a branch owned by the decider, and a loop that keeps the whole thing alive.

Development happens in an ordinary directory with ordinary command-line tools. You can write the workflow yourself or ask a coding agent such as Claude Code or Codex to help. This tutorial shows both paths and points out where human input is needed.

1 What you are building



After sending or discarding the reply, the service waits for the next message and repeats. The loop keeps it running as a service.

Concretely, you want this to happen:

Incoming message

Could we move our meeting to Thursday afternoon?

Proposed reply

Thursday afternoon works for me. How about 3pm?

`[ Confirm ] [ Decline ]`

Two participants exchange messages, and one local action asks a person. What happens next depends on that person's answer. ZipperGen makes this coordination explicit and readable.

What you need

- Python 3.11 or newer and ZipperGen installed.
- The first run uses a mock model and terminal approval.
- For the Telegram section, a bot token, if you want the approval to arrive on your phone.

2 Create the project

```
mkdir email-approval && cd email-approval
zippergen init
```

```
ZipperGen project: email-approval
  zippergen.toml     created
  specification.md   created
  AGENTS.md          created
  CLAUDE.md          created
```

zg is short for zippergen

Both commands are installed and do the same thing. The rest of this tutorial uses `zg`, because the commands you type often are the ones worth shortening.

The command creates the four files above. Configuration comes later, when you select a model and connector.

- `zippergen.toml` is the project's visible configuration: every non-secret choice, including the answers you give when deploying. It is committed, while credentials stay in ZipperGen's private store.
- `specification.md` says what the workflow is meant to do, in prose.
- `AGENTS.md` tells a coding agent to run `zippergen skill` before touching workflow code.
- `CLAUDE.md` points Claude Code at the same shared instructions.

It also creates `.zippergen/`, which holds the generated identity for this checkout and links it to its credentials. Git ignores this directory. A clone therefore selects its private workspace from its local path and uses its own credentials. Keep `zippergen.toml` for visible configuration and leave `.zippergen/` to ZipperGen.

3 Describe what you want

The next section shows the workflow code. You can write it yourself or describe what you want in ordinary language. Open a coding agent in the project directory:

```
claude  # or: codex
```

Then ask:

```
> Build a ZipperGen workflow that watches a mailbox directory, asks an LLM
  to draft a short reply to each new message, and asks me to approve it
  before it is sent. It should keep running and wait for the next message.
```

```
  Use plain .txt files directly inside mailbox/, with the entire file as the
  message body. A file containing only "Can we meet on Thursday" is a complete
  message. Use mailbox/ as the local default. Verify that "zg run --llm mock"
  reaches approval directly with this exact layout.
```

The agent reads `AGENTS.md`, which directs it to run `zg skill`. The skill explains how to write and check a workflow and which boundaries to preserve. The agent writes the files, then checks its work with the same commands you would use:

```
I updated specification.md and created workflow.py.

  $ zg validate
  Workflow email_approval: valid

Two participants, Mailbox and Writer, and one decision that Mailbox owns:
whether to send the draft. Ask me to change anything.
```

You describe what you want. The agent edits ordinary files and checks them with ZipperGen's commands. The workflow and its checks remain visible in the project directory for you to inspect or edit.

Where the skill comes from

`zg skill` prints instructions that ship inside the package, so they match the version you have installed. They are ordinary prose in the repository. After upgrading ZipperGen, start a new Claude or Codex session, or ask the current session to run `zg skill` again.

4 What comes out

The workflow is visible, readable Python:

```
@workflow
def email_approval() -> int:
    Mailbox: message = next_unread_message()
    while message @ Mailbox:
        Mailbox(message) >> Writer(message)
        Writer: draft = draft_reply(message)
        Writer(draft) >> Mailbox(draft)
        Mailbox: approved = approve_reply(draft)
        if approved @ Mailbox:
            Mailbox: handled = send_reply(draft, handled)
        else:
            Mailbox: handled = discard(handled)
        Mailbox: message = next_unread_message()
    return handled @ Mailbox
```

Read it top to bottom and you have the entire message order. Five things are worth noticing.

`Mailbox(message) >> Writer(message)` is a message. The left side sends, the right side receives, and the names in brackets say which variables carry the value.

The line beginning `Writer:` is an action performed by one participant. `draft_reply` is an `@llm` action, which is a model call with a prompt and a declared output.

`Mailbox: approved = approve_reply(draft)` is a `@human` action. It stops and waits for a person.

`if approved @ Mailbox` is a decision, and the `@ Mailbox` says who makes it. That annotation is the important one, and the next section shows why.

`while message @ Mailbox` is a loop, owned the same way. The `Mailbox` decides each round whether there is another message to handle. `next_unread_message` waits until there is one, so in practice that decision is always yes and the workflow keeps running.

Mailbox handling stays local

Mailbox handling involves waiting for new mail, choosing between polling and blocking, and marking each message as handled. All of it sits behind `next_unread_message`, whose contract is one sentence: *return the next message, waiting if there is none.* Here it takes the oldest `.txt` file from `mailbox/` and renames it `.done`, making each message one-shot even across a crash. The same coordination works with a real inbox.

5 Check it yourself

Everything the agent ran, you can run:

```
zg validate
```

```
Workflow email_approval: valid
OK   lifelines: 2 unique lifeline(s): Writer, Mailbox
OK   variable availability: every participant reads only values definitely available
    locally
OK   projection Writer: WhileRecvStmt
OK   projection Mailbox: SeqStmt
OK   deployment declaration: 1 field(s), 0 package(s), 0 setup step(s)
OK   workflow inputs: none, the run starts without setup questions
OK   canonical rendering: global and local code views rendered successfully
```

`validate` loads the module, projects the protocol for every participant, and renders it back. If it passes, the workflow is well formed and the deadlock-freedom guarantee applies to it.

You and the agent use the same commands. You supply credentials yourself. The rest of this tutorial moves between asking the agent for prose-level changes and running short commands directly.

6 Inspect the projected programs

The workflow has one decision and `Mailbox` owns it. Ask the agent for the `Writer`'s projected program:

```
> What does the Writer actually run?
```

```
  $ zg show --agent Writer

  @role('Writer')
  def email_approval__Writer() -> None:
      while recv_decision('Mailbox'):
          message = recv('Mailbox')
          draft = draft_reply(message)
          send('Mailbox', draft)

The Writer receives the loop decision each round and drafts one reply.
Mailbox owns the later approval decision, so it appears only in Mailbox's
program.
```

Run it yourself and you get the same projection. Now compare the `Mailbox`, which owns both the loop and the decision:

```
zg show --agent Mailbox
```

```
@role('Mailbox')
def email_approval__Mailbox() -> int:
    message = next_unread_message()
    while message:
        send_decision('Writer', True)
        send('Writer', message)
        draft = recv('Writer')
        approved = approve_reply(draft)
        if approved:
            handled = send_reply(draft, handled)
        else:
            handled = discard(handled)
        message = next_unread_message()
    else:
        send_decision('Writer', False)
    return handled
```

The Writer has no approval branch

ZipperGen generated both local programs from the workflow. Put them side by side: `Mailbox` has code for each of its two control structures, while `Writer` has code for only one of them. The `Writer` participates in the loop, so `Mailbox` sends it the loop decision every round and sends the final false decision on exit. The `send_decision` in the `else` carries that final decision.

The approval belongs entirely to `Mailbox`. The `Writer` participates in neither branch, so projection erases that decision from its program. Its execution proceeds independently of the answer.

Hand-written multi-agent systems often fail here when one agent waits for a message that another branch omits. Projection derives the required control recipients from the workflow.

These projection rules carry a formal guarantee. For well-formed workflows, complete executions of the projected local programs produce exactly the behaviours of the global protocol, up to generated control messages. Every finite projected execution prefix can also be extended to a complete one. The second property is the formal deadlock-freedom guarantee.

7 Run it

First put something in the mailbox. A message is just a text file:

```
mkdir -p mailbox
echo "Can we meet on Thursday" > mailbox/01.txt
```

Put the files directly in `mailbox/`. A plain one-line file is the message body, and headers such as `From` and `Subject` are optional.

Validation also makes unexpected setup inputs visible. For this workflow it must report:

```
OK   workflow inputs: none, the run starts without setup questions
```

The mock backend works without a model key and answers model calls with a placeholder:

```
zg run --llm mock
```

```
No real model is in use: every participant answers with the mock. Assign one with 'zg
    model assign TARGET NAME'.

REQUEST · Mailbox

Proposed reply:

[draft_reply:draft]

Send this reply? [y/n]: y
✓ Mailbox · reply sent
```

The process now waits for the next message. Drop another file into `mailbox/` from a second terminal and it wakes up and handles that one too. Answering `n` takes the other branch.

`mailbox/01.txt` is now `mailbox/01.done`. That rename is what stops a message being answered twice, including after a crash.

Stop it with `Ctrl-C`. For a run that ends by itself, add two new messages and give it a budget:

```
echo "Please send the notes by Friday" > mailbox/02.txt
echo "Can we talk next week?" > mailbox/03.txt
printf 'y\nn\n' | zg run --llm mock --option max_messages=2
```

```
{"result": 1}
```

The two answers approve the first reply and decline the second. The result counts replies sent, so it is one.

> **Testing a specific branch**
>
> `mock` supplies placeholder answers for model actions. A `@human` action still reads an answer from the terminal or its assigned connector. Pipe `y` or `n`, as above, when a test needs a fixed human decision. Use `--llm scripted:replies.json` when a model answer must be fixed.

8 Use a real model

Create a named configuration and assign the Writer to it:

```
zg provider configure openai-main openai
zg provider set-credential openai-main
zg model configure writer openai-main gpt-4o-mini
zg model assign Writer writer
zg model
zg run
```

The model identity and assignment are written to `zippergen.toml`, so they travel with the project. The credential command reads the API key through a hidden prompt and saves it privately for the named provider connection on this computer. You can instead export the key before running the command. Several participants can share the same named configuration, and one exact action can be assigned with a target such as `Writer.draft_reply`. `zg run --llm mock` remains useful as a temporary override.

Temperature is portable model behavior. An individual `@llm(temperature=...)` declaration overrides the named configuration. Zero reduces sampling variation but does not guarantee identical

answers from a hosted model. If a selected model removed sampling controls, ZipperGen reports that incompatibility.

When working by hand, you can also run `zg model configure` and `zg model assign` with no values. ZipperGen then asks the questions and shows the available choices. It asks for the provider connection and model separately. Full commands remain better for scripts and coding agents.

9 Approval on your phone

So far the approval has appeared in whichever terminal is running the workflow. Terminal approval suits development. An unattended workflow needs another route, so ask for the change:

```
> I want the approval to arrive on Telegram instead.
```

```
The Mailbox's @human action already defines the approval, so the workflow stays
the same. I updated the project configuration that controls where the question
is delivered.

I also updated specification.md to say approvals go to Telegram, and
zg validate still passes.

Finish the private setup in your terminal, where ZipperGen can read the chat id
and bot token through interactive prompts. Run:

    zg provider configure approval-bot telegram
    zg provider set-credential approval-bot
    zg connector configure approval-chat approval-bot
    zg connector assign Mailbox approval-chat

Tell me when that is done and I will check the configuration.
```

The agent can display the complete stored configuration offline with `zg config`. Use `zg check` when you want ZipperGen to verify live readiness. It contacts configured providers and may send a small model request.

This division keeps workflow edits with the agent and credential entry in your terminal. The hidden prompt keeps the token out of the agent's conversation, shell history, and repository.

The saved token lives in a file under `ZIPPERGEN_HOME` readable by your user. A coding agent running as that user with shell access can therefore read it. For a hard capability boundary, give the agent a separate account or an environment with a separate credential store.

Coding agent

```
> I want the approval to arrive
  on Telegram instead.

Updated the specification.
zg validate passes.

Telegram setup needs your terminal:

    zg provider configure
      approval-bot telegram
    zg provider set-credential
      approval-bot
    zg connector configure
      approval-chat approval-bot
    zg connector assign Mailbox
      approval-chat
```

Your terminal

```
$ zg provider configure
    approval-bot telegram
$ zg provider set-credential
    approval-bot

Telegram bot token (hidden):
Saved the token for this computer.

$ zg connector configure
    approval-chat approval-bot

Telegram chat id: 12345678
Saved configuration approval-chat.

$ zg connector assign Mailbox
    approval-chat
Mailbox will be asked through approval-chat.
```

Here `approval-chat` is a user-chosen name. The provider commands name one Telegram bot and

save its token on this computer. The connector configuration says *which* chat. The assignment says *whose action* is routed there: `Mailbox`, whose `@human` action is the approval.

The chat destination and bot credential are stored separately:

chat id	written to `zippergen.toml`, committed, shared
bot token	entered through a hidden prompt, stays on this computer

A colleague who clones the project receives the chat id and supplies their own token through the hidden prompt.

10 Runs that survive interruption

Add `--durable` to a foreground run to record each step and resume from its committed position after an interruption:

```
echo "Could we meet on Tuesday?" > mailbox/04.txt
zg run --durable --llm mock
```

```
Workflow email_approval: valid
No real model is in use: every participant answers with the mock. Assign one with 'zg
    model assign TARGET NAME'.
Run email_approval-20260831-132719-612865000

REQUEST · Mailbox

Proposed reply:
...
```

Press Ctrl-C while it waits for your approval, then:

```
zg run inspect --agent Writer
zg run --resume
```

The model result and control position committed before the interruption are preserved, so this run resumes at the approval and skips another draft. A crash during an external call can still repeat that call if its result had not yet committed.

To abandon the selected run, stop it with Ctrl-C and use `zg run reset`. This permanently discards its record and SQLite state and clears the current run. Add `--archive` when you want a recoverable private copy. Start another run with `zg run --durable`. Starting a new durable run also discards an older selected development run.

While a durable run is active in one terminal, another terminal can keep its program position open and refresh it once per second:

```
zg run inspect --watch --agent Writer
```

Pressing Ctrl-C in this view closes the view and leaves the run active.

For a long-running workflow, continue a foreground durable run with `zg run --resume`. A deployment restarts through its service manager. In both cases, committed state preserves the workflow position across an ordinary stop.

11 Deploy it

The ordinary deployment command prepares, checks, and starts the service. For this example it first asks for the mailbox directory. Enter the project directory you used above, such as `mailbox`. ZipperGen records its absolute path so the deployed service continues watching that directory from outside its immutable source bundle:

```
zg deploy
```

Every model, assistant CLI, and connector must pass a readiness probe before the service starts:

```
zg deploy logs
zg deploy status
zg deploy inspect --watch
zg deploy trace --follow
```

`zg deploy status` distinguishes freshness of the ZipperGen runtime from freshness of the workflow source bundle. A stale warning means that the immutable service is still running its previous snapshot. Run `zg deploy stop`, then bare `zg deploy`, to publish and start the update. Status also names an external action currently in flight, with its elapsed time, and the last terminal lifeline failure. Trace begins with service and runtime freshness and identifies a stopped deployment explicitly. `inspect --watch` redraws current state. `trace --follow` keeps the event table and appends newly committed rows. A live action first appears as *start*, then *done* or *failed*. Only a static trace calls a start *incomplete* when no terminal event was recorded. Use status to decide what is running now.

Use `zg deploy --no-start` when you deliberately want to prepare and review a stopped deployment before a later `zg deploy start`. The usual command is bare `zg deploy`.

The trace keeps the newest 10,000 events by default. The budget counts FIFO rows, not time or bytes. Frequent routine polls can evict an incident, and events carrying whole emails can make the same count use much more disk. Measure the real workflow before choosing `zg deploy --history-keep N`. Very large budgets also make the periodic writer-path prune more expensive.

`zg deploy reset --yes` is different from stopping. It archives and replaces all durable execution state, including connector progress, and always leaves the service stopped. A later start begins the workflow again and may re-read mail already handled.

A deployment you no longer want:

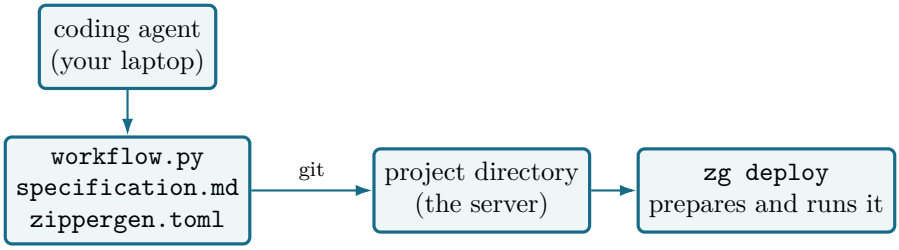
```
zg deploy remove
```

Its profile, durable store and log are moved to ZipperGen's trash directory for recovery. Deploying again builds a new deployment with an empty store. Removal also deletes its credentials, which must be entered again. A future deployment rebuilds the managed environment, service files, and bundled source. `--purge` also deletes the archived profile, store, and log.

12 Put it on a server

Everything above works on your laptop. Continue when you want the workflow to keep running after you close it.

ZipperGen deploys to a machine you control. Copy the project there, install ZipperGen, and run the same commands you used locally.



Only the project files cross to the server. The coding agent stays on your laptop after writing the workflow. The server runs ZipperGen with those project files and its own credentials.

On an ordinary Linux machine:

```
git clone https://github.com/me/email-approval.git && cd email-approval
python3 -m venv .venv && source .venv/bin/activate
pip install zippergen

mkdir -p mailbox
zg provider set-credential openai-main
zg provider set-credential approval-bot

zg deploy
```

The clone supplies the model and connector routing. The two credential commands read the OpenAI key and Telegram bot token through hidden prompts and store them outside the project. When `zg deploy` asks for the mailbox directory, enter `mailbox`.

The last command prepares, checks, and starts the deployment. It probes the model and connector first, so a mistyped token is reported before the service begins handling approvals. On Linux it installs a small `systemd` user service that restarts the workflow if it fails, and enables that unit for autostart. A server account also needs its user manager to survive logout and start at boot. If local policy permits it, enable linger once:

```
loginctl enable-linger "$USER"
zg deploy check
```

The check reports both systemd autostart and linger. Before leaving the service unattended, log out once and reboot once, checking `zg deploy status` after each. ZipperGen writes the unit file. `zg deploy status`, `zg deploy logs`, and `zg deploy stop` manage it from here.

The workflow's continuous loop makes it a natural fit for a server. Once deployed, your laptop can close.

The mailbox here is a directory, which is enough to see the shape. Point `next_unread_message` at a real inbox and the same coordination remains. `examples/inbox_triage.py` is that workflow, reading Gmail and writing to a spreadsheet.

13 What you have

`specification.md`	what it should do, in prose, maintained by you and your agent
`workflow.py`	the coordination, as visible Python
`zippergen.toml`	which model, which chat, committed, no secrets
`mailbox/`	the local inbox watched by this example

The table lists the workflow's three working files and its mailbox directory. The project also retains `AGENTS.md` and `CLAUDE.md` for coding agents. Each machine stores its model key and bot token privately.

The `Writer`'s projected program contains the loop where it has work and omits the approval owned by `Mailbox`. You wrote one protocol, and ZipperGen derived what each participant runs in a way that cannot deadlock. As the workflow grows, with more participants, more loops and parallel regions, that keeps the coordination manageable.

14 Where to go next

- `examples/diagnosis.py` has two reviewers who argue until they agree. A loop, and a decision inside it.
- `examples/inbox_triage.py` reads a real mailbox and records to a real spreadsheet, runs as a service.
- *Development and deployment guide* is the longer reference.
- `zippergen skill` prints what your coding agent is following. Worth reading once yourself.